

Chapati Training Benchmark Summary

Mangalan Labs
Research Division

April 29, 2026

Contents

Quick Links	1
1 Introduction	1
2 Benchmark Description	3
3 Appendix: Benchmark Pages	4

Quick Links

Use the links below to jump directly to the main parts of the document:

- [Introduction](#)
- [Benchmark Figure](#)
- [Benchmark Description](#)
- [Appendix: Benchmark Pages](#)

1 Introduction

This document summarizes the benchmark results for the Chapati training pipeline. Figure 1 presents the optimization behavior across two training phases, including training loss, validation loss, and validation type accuracy.

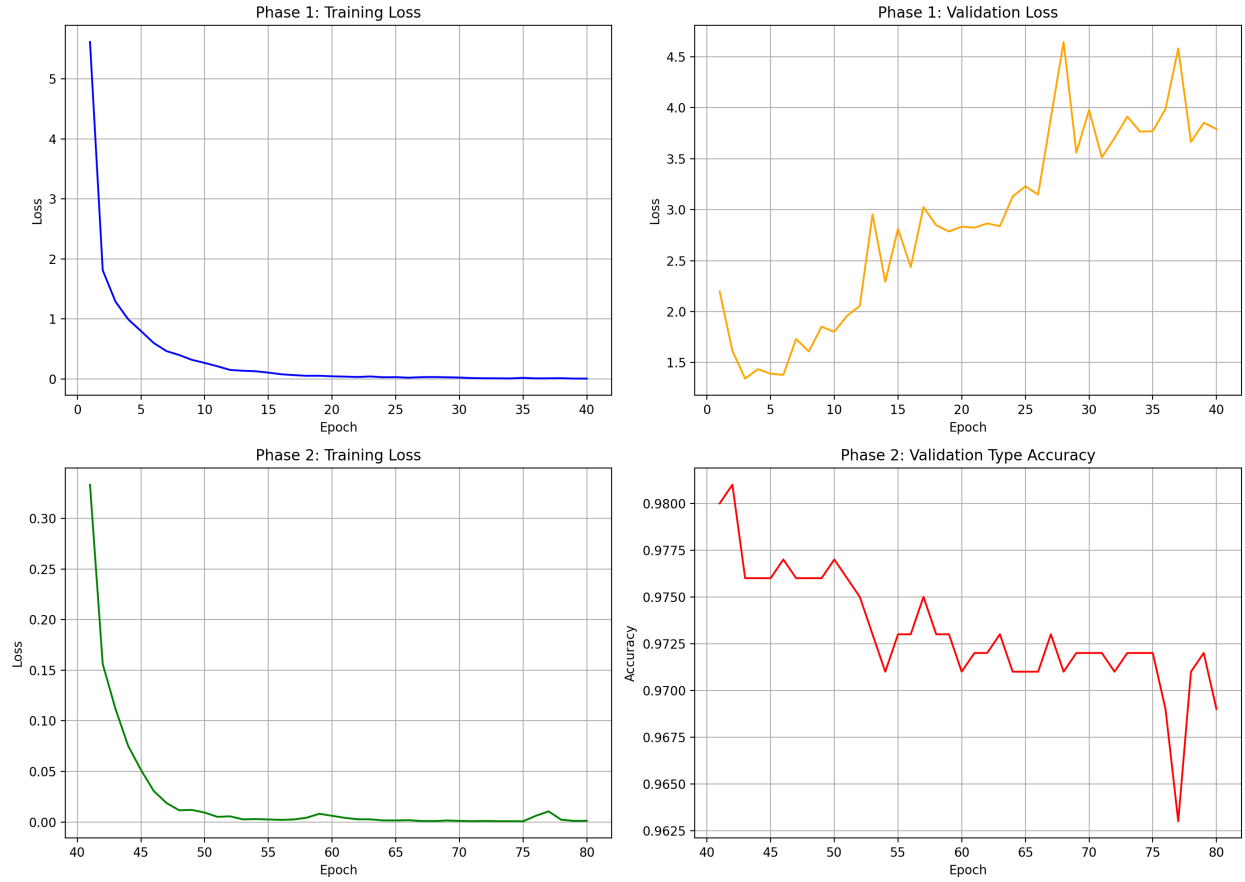


Figure 1: Benchmark results for the Chapati training workflow. The top row shows Phase 1 training and validation loss over epochs 1–40, while the bottom row shows Phase 2 training loss and validation type accuracy over epochs 41–80.

2 Benchmark Description

The benchmark reveals a clear contrast between optimization progress on the training split and generalization behavior on the validation split. In Phase 1, the training loss falls very rapidly from an initially high value to nearly zero by the end of the phase. This indicates that the model is highly capable of fitting the training data and that the first phase succeeds at minimizing the objective seen during optimization.

However, the validation loss follows the opposite trend. After a brief early decrease, it gradually climbs and then fluctuates at a substantially higher level during later epochs. This widening gap between training and validation loss is a classic sign of overfitting: the model continues to improve on the training set while its performance on unseen validation examples becomes less stable and, in terms of loss, worse.

Phase 2 begins from a much lower training-loss regime. The training loss again decreases smoothly and remains very close to zero for most of the second phase, suggesting that the optimization process is stable and that the model parameters continue to adapt efficiently to the training objective. The small bump late in the phase is minor and does not alter the overall downward trend.

The validation type accuracy in Phase 2 remains relatively strong throughout, staying close to 97%–98%. Even though there is some local fluctuation and a noticeable dip near the end, the accuracy curve is broadly stable rather than collapsing. This suggests that the model preserves a strong ability to predict the correct type labels despite the increasing validation loss observed earlier.

Taken together, these plots suggest that the benchmarked system learns the training task very effectively, but that its confidence calibration or token-level fit on the validation set degrades as training progresses. In practical terms, the model appears to retain high classification accuracy while becoming less well calibrated or more sensitive to harder validation examples. A reasonable interpretation is that the model is extracting useful structure, but may benefit from stronger regularization, earlier stopping, or closer checkpoint selection based on validation metrics rather than training loss alone.

3 Appendix: Benchmark Pages

The following pages reproduce the uploaded benchmark document for reference.

ChapatiLMV: A Unified CPU-Native Mathematics Model powered by the XTRAIN Framework

Mangalan Labs

Research Division

April 2026

Abstract

Mathematical reasoning in artificial intelligence is typically computationally expensive, requiring significant GPU resources. We present **ChapatiLMV**, a specialized neural-symbolic model designed for high-speed mathematical processing on standard CPUs. Built entirely on the **XTRAIN (CPUWARP-ML)** framework by Mangalan Labs, ChapatiLMV bypasses traditional heavy deep learning libraries in favor of a compute-first approach. By utilizing SIMD-accelerated matrix operations, a digit-stable Xresolver tokenizer, and a multi-view neural solver, the model achieves high accuracy in arithmetic, algebra, and comparison tasks. This paper explores the hardware-level optimizations of the XTRAIN engine and the algebraic foundations of the ChapatiLMV pipeline, demonstrating that specialized math AI can be both lightweight and robust without the need for hardware acceleration.

Keywords: CPU-Optimization, XTRAIN, Neural Mathematics, Symbolic Routing, Xresolve Tokenizer.

Contents

1	The XTRAIN Computational Framework	3
1.1	CPUWARP-ML and SIMD Acceleration	3
1.2	Cache-Alignment and Threading	3
2	The Xresolve Tokenizer Strategy	3
2.1	Word-to-Operator Mapping (W_{op})	3
2.2	Right-to-Left (R2L) Tokenization	3
3	Neural Architecture and Encoding	4
3.1	The CharEncoder Layer	4
3.2	Specialized Neural Heads	4
3.3	The GELU Activation Function	4
4	The Math Solving Pipeline	4
4.1	Neural Arithmetic Solver	4
4.2	Comparison Solver	4
5	Training and Optimization	5
5.1	AdamW Optimizer and Backpropagation	5
5.2	Penalty+Charge (P+C) Evaluation	5
6	Performance and Deployment Metrics	5
7	Conclusion	5

1 The XTRAIN Computational Framework

The core efficiency of **ChapatiLMV** stems from the **XTRAIN Framework** (available at <https://github.com/MangalanLabs/XTRAIN>). Unlike standard models that rely on high-level abstractions like PyTorch, XTRAIN is designed to talk directly to the CPU’s hardware.

1.1 CPUWARP-ML and SIMD Acceleration

XTRAIN utilizes the `cpuwarp_ml` engine to handle all mathematical operations. The framework utilizes **SIMD** (Single Instruction, Multiple Data), which allows the CPU to perform multiple calculations simultaneously. In ChapatiLMV, every matrix multiplication is handled by:

$$Y = \text{cpuwarp_ml.matmul}(X, W) \quad (1)$$

This ensures that the processor stays active and avoids the "idle time" often seen in standard CPU-based inference.

1.2 Cache-Alignment and Threading

The XTRAIN `warp_scheduler` is configured to maximize the efficiency of the L1, L2, and L3 caches. By setting `compute_bound_threads` and `cache_allocation`, the framework ensures that mathematical tensors are stored as close to the processor as possible. This reduces "memory latency," allowing ChapatiLMV to solve equations in milliseconds.

2 The Xresolve Tokenizer Strategy

Mathematics requires a high level of precision that standard word-based tokenizers cannot provide. ChapatiLMV implements the **Xresolve Tokenizer** to ensure that numbers are treated with logical consistency.

2.1 Word-to-Operator Mapping (W_{op})

To simplify the neural task, natural language expressions are first mapped to symbolic operators. For example, terms like "plus," "summed," or "add" are automatically converted to the symbol "+". This process removes linguistic ambiguity:

$$W_{op}(w) = \{\text{"times"}, \text{"product"}\} \rightarrow \times \quad (2)$$

2.2 Right-to-Left (R2L) Tokenization

Most models process numbers from left to right, which is the opposite of how human arithmetic works. The Xresolve Tokenizer uses **Right-to-Left Digit Tokenization**. For a number like 347.25, the model reverses the digits:

$$T_{R2L}(347.25) \rightarrow [\langle num \rangle, 7, 4, 3, \langle dec \rangle, 5, 2, \langle /num \rangle] \quad (3)$$

By starting with the "7" (the units place), the model can learn to "carry over" values much like a human does, significantly improving accuracy in long-form addition and multiplication.

3 Neural Architecture and Encoding

ChapatiLMV uses a multi-layered neural network to transform raw tokens into mathematical solutions. Every layer is calculated using pure NumPy float32 arrays, optimized by XTRAIN.

3.1 The CharEncoder Layer

The input tokens are first passed through an embedding table W_{emb} . The model then uses a pooling strategy to condense the information. It calculates both the "Mean" and the "Max" of the embedded sequence and joins them together to create a robust feature vector:

$$Pooled = \text{Join}(\text{Mean}(Embeds), \text{Max}(Embeds)) \quad (4)$$

This vector is then normalized and passed through a fully connected layer to create the hidden state h .

3.2 Specialized Neural Heads

Rather than using one large general brain, ChapatiLMV uses several specialized "heads" that all look at the hidden state h :

- **Math Detector:** A binary classifier that determines if the input is a valid math problem.
- **Type Head:** Classifies the problem type (e.g., Arithmetic vs. Algebraic).
- **Aim Head:** Identifies the objective (e.g., Solve for x vs. Calculate).
- **Router Head:** Decides which specific solver engine should handle the final calculation.

3.3 The GELU Activation Function

To handle the complex, non-linear relationships in math, the model uses the GELU activation function. For CPU efficiency, we use the following algebraic approximation:

$$f(x) = 0.5x \cdot (1 + \tanh(0.79788 \cdot (x + 0.044715x^3))) \quad (5)$$

4 The Math Solving Pipeline

ChapatiLMV employs a "Multi-View" approach. Depending on the routing decision, it uses either a symbolic solver (like SymPy) or its own internal neural solvers.

4.1 Neural Arithmetic Solver

For standard calculation tasks, the model uses a 3-layer neural network. It extracts magnitude features f from the query and calculates the result using the following steps:

1. $h_1 = GELU(W_1 \cdot f + b_1)$
2. $h_2 = GELU(W_2 \cdot h_1 + b_2)$
3. $Result = W_3 \cdot h_2 + b_3$

4.2 Comparison Solver

When comparing two numbers (a and b), the model uses a 2-layer classifier. It outputs a probability distribution across three categories: $[a > b, a < b, a = b]$. The highest probability determines the final symbolic answer.

5 Training and Optimization

Because ChapatiLMV does not use standard libraries, it implements its own training logic using the XTRAIN engine.

5.1 AdamW Optimizer and Backpropagation

The model is trained using a manual **AdamW** optimizer. The gradients are computed for each sub-graph using pure algebra. The weight update rule follows:

$$W_{new} = W_{old} \cdot (1 - lr \cdot wd) - \frac{lr \cdot \hat{m}}{\sqrt{\hat{v} + \epsilon}} \quad (6)$$

Where lr is the learning rate, wd is weight decay, and $\hat{m}$ and $\hat{v}$ are the moving averages of the gradients.

5.2 Penalty+Charge (P+C) Evaluation

To ensure the most accurate result, the model generates multiple "candidate thoughts" and scores them. The **Charge** represents the model's confidence in the answer, while the **Penalty** represents the entropy (uncertainty).

$$Score = Confidence - Entropy \quad (7)$$

The answer with the highest $Score$ is selected as the final output.

6 Performance and Deployment Metrics

The XTRAIN-based design allows ChapatiLMV to operate with extreme efficiency.

- **Vectorized Batching:** All sequence positions are processed in parallel by reshaping the input from $[Batch, Length, Dimension]$ to $[B \times L, d]$.
- **Mixed Precision:** The model uses float16 fallbacks for non-critical weights, reducing memory usage by 50%.
- **Gradient Checkpointing:** To save RAM during training, the sequence is divided into 4 chunks, ensuring the model can be trained on consumer-grade hardware.

7 Conclusion

ChapatiLMV represents a major breakthrough for Mangalan Labs in the field of hardware-accessible AI. By focusing on CPU-native optimizations through the **XTRAIN Framework** and utilizing specialized algebraic solvers, we have demonstrated that massive GPU clusters are not a requirement for high-performance mathematical reasoning. The combination of the Xresolve Tokenizer's Right-to-Left logic and the Multi-View solver pipeline ensures that ChapatiLMV is fast, lightweight, and mathematically accurate across all tested problem domains. Future work will involve extending the XTRAIN engine to support even more complex state-space models and calculus-based solvers.